user-service

JUnit Test Suite Documentation

Comprehensive reference for all unit tests

Overview

This document describes every JUnit 5 unit test in the user-service test suite. Each test is written with Mockito and uses StepVerifier (Project Reactor) for reactive stream assertions. Tests are tagged with @Tag("unit") and are fully isolated — no Spring context, no database, no Kafka, no gRPC server is started.

The test suite covers the following areas:

Test Class	Layer	Responsibility
`UserServiceImplTest`	**Service**	Profile, social graph, search, verification
`FollowerServiceImplTest`	**Service**	Follow / unfollow, social graph reads
`BlockServiceImplTest`	**Service**	Block / unblock, bidirectional status
`AccountHistoryServiceImplTest`	**Service**	Record and retrieve account history
`FollowRequestServiceImplTest`	**Service**	Create, respond and list pending requests
`UserSettingsServiceImplTest`	**Service**	Read, default and update user settings
`UserHandlerTest`	**Web (HTTP)**	REST endpoints via WebTestClient
`UserGrpcServiceTest`	**gRPC**	gRPC method contracts
`UserEventPublisherTest`	**Kafka**	Event publishing to Kafka topics
`DtoValidationTest`	**Validation**	Bean Validation constraints on DTOs

1. UserServiceImplTest

Class — service/UserServiceImplTest.java

Tests UserServiceImpl, the central service that orchestrates profile management, the social graph and identity verification. The service is instantiated manually so all dependencies are Mockito mocks.

Setup

A testUser (username: "alice") and a UserPrincipal wrapping her UUID are created before each test. LockService is stubbed to execute the supplier immediately.

Test Cases

Test Method	What It Verifies	Expected Outcome
`getByUsername – found`	Given an existing username, the service loads the user and returns a UserResponse	Returns UserResponse
`getByUsername – not found`	Given an unknown username, the service propagates the empty Mono as ResourceNotFoundException	throws ResourceNotFoundException
`getById – found`	Given an existing UUID, returns the mapped UserResponse	Returns UserResponse
`getById – not found`	Given an unknown UUID, throws ResourceNotFoundException	throws ResourceNotFoundException
`updateProfile – partial update`	Only non-null fields in UpdateProfileRequest are applied; the save arg is verified via argThat	Returns updated UserResponse
`updateProfile – user not found`	If the user does not exist, throws without calling save	throws ResourceNotFoundException
`updateAvatar – success`	Saves the new avatarUrl and publishes a user.avatar.changed event	Event published, returns UserResponse
`updateAvatar – user not found`	If the user does not exist, throws	throws ResourceNotFoundException
`updateBackground – success`	Saves the new backgroundUrl and publishes a user.background.changed event	Event published, returns UserResponse
`updateBackground – user not found`	If the user does not exist, throws	throws ResourceNotFoundException
`updateSettings – valid JSON`	Serialises the settings map and saves the updated user	Returns UserResponse
`follow – self follow`	Following yourself is rejected without any DB query	throws ConflictException

Test Method	What It Verifies	Expected Outcome
`follow – target not found`	Following a non-existing user throws	throws ResourceNotFoundException
`follow – public user`	Following a public account delegates to FollowerService.follow; FollowRequestService is never called	Delegates to FollowerService
`follow – private user`	Following a private account creates a follow request; FollowerService.follow is never called	Delegates to FollowRequestService
`unfollow`	Delegates to FollowerService.unfollow with the correct IDs	Delegates to FollowerService
`getFollowers`	Delegates to FollowerService.getFollowers and returns the PageResponse	Returns PageResponse
`getFollowing`	Delegates to FollowerService.getFollowing and returns the PageResponse	Returns PageResponse
`getFollowRequests`	Delegates to FollowRequestService.getPending with the principal's UUID	Returns PageResponse
`respondFollowRequest`	Delegates to FollowRequestService.respond with correct args	Completes
`getHistory`	Delegates to AccountHistoryService.getHistory with limit 50	Returns PageResponse
`searchUsers – short query (<2 chars)`	Returns an empty page immediately without querying the DB	Returns empty page
`searchUsers – null query`	Null is treated the same as a short query; no DB call	Returns empty page
`searchUsers – valid query`	Queries DB and maps results to a UserResponse page	Returns populated page
`submitVerification – already pending`	If a PENDING verification exists, rejects the new request	throws ConflictException
`submitVerification – user not found`	If no pending verification but user missing, throws	throws ResourceNotFoundException
`submitVerification – success`	Saves a new Verification entity with PENDING status	Verification saved

2. FollowerServiceImplTest

Class — service/FollowerServiceImplTest.java

Tests FollowerServiceImpl, which manages the follow relationship between users. LockService is stubbed to execute each supplier synchronously, removing any Redis dependency.

Test Cases

Test Method	What It Verifies	Expected Outcome
`follow – already following`	If the follow relationship already exists, throws without saving	throws ConflictException
`follow – new follow`	Saves a Follower row, increments the target's followerCount (+1) and the follower's followingCount (+1), then publishes a user.followed event	Row saved, counts incremented, event published
`unfollow – not following`	If the relationship does not exist, throws	throws ConflictException
`unfollow – existing follow`	Deletes the follow row, decrements both counters (-1), publishes a user.unfollowed event	Row deleted, counts decremented, event published
`isFollowing – exists`	Returns true when the relationship is present	Returns true
`isFollowing – absent`	Returns false when the relationship is absent	Returns false
`getFollowers`	Loads Follower records for the target, resolves each follower's User, maps to UserResponse page	Returns mapped page
`getFollowing`	Loads Following records for the requester, resolves each followed User, maps to UserResponse page	Returns mapped page

3. BlockServiceImplTest

Class — service/BlockServiceImplTest.java

Tests BlockServiceImpl, which manages user blocks. The service is guarded by a distributed lock; the lock is stubbed to be transparent in tests.

Test Cases

Test Method	What It Verifies	Expected Outcome
`block – self block`	Attempting to block yourself is rejected immediately without querying the DB	throws ConflictException
`block – already blocked`	If the block already exists, throws	throws ConflictException
`block – new block`	Saves a UserBlock entity with the correct blocker and blocked IDs	Block saved
`unblock – no block`	Attempting to remove a non-existent block throws	throws ResourceNotFoundException
`unblock – existing block`	Deletes the block row by blocker/blocked composite key	Block deleted
`isBlocked – true`	Returns true when blockRepo confirms the block exists	Returns true
`isBlocked – false`	Returns false when blockRepo confirms no block	Returns false
`isEitherBlocked – A blocks B`	Returns true as soon as the first direction confirms a block, without checking the reverse	Returns true
`isEitherBlocked – B blocks A`	When A→B is absent but B→A exists, still returns true	Returns true
`isEitherBlocked – neither`	Returns false only when both directions have no block	Returns false

4. AccountHistoryServiceImplTest

Class — service/AccountHistoryServiceImplTest.java

Tests AccountHistoryServiceImpl, which records security events (logins, profile changes, etc.) and retrieves them for a user.

Test Cases

Test Method	What It Verifies	Expected Outcome
`record – valid input`	Saves an AccountHistory with userId, action, detail and ip exactly as provided; verified via argThat	History saved
`record – null detail`	When detail is null the entity is still saved with a null detail field	History saved with null detail
`getHistory – records exist`	Returns a PageResponse containing the history items in the order returned by the repository	Returns populated page (2 items)
`getHistory – no records`	Returns an empty PageResponse when the repository returns nothing	Returns empty page

5. FollowRequestServiceImplTest

Class — service/FollowRequestServiceImplTest.java

Tests FollowRequestServiceImpl, which handles follow requests for private accounts. The respond method accepts or rejects a pending request and has several authorization guards.

Test Cases

Test Method	What It Verifies	Expected Outcome
`create – already pending`	If a PENDING request already exists between the same pair, throws	throws ConflictException
`create – new request`	Saves a FollowRequest with status PENDING	Request saved
`respond – invalid action`	Actions other than APPROVE or REJECT are rejected before any DB call	throws ValidationException
`respond – request not found`	If the request ID does not exist, throws	throws ResourceNotFoundException
`respond – wrong target user`	Only the request's target can respond; a different user gets ForbiddenException	throws ForbiddenException
`respond – already actioned`	A request whose status is not PENDING cannot be actioned again	throws ConflictException
`respond – APPROVE`	Creates a Follower row, increments both counters, publishes user.followed event, and updates status to APPROVED	Follower saved, event published
`respond – REJECT`	Updates status to REJECTED; does not create a Follower row or publish an event	Status updated only
`getPending – requester found`	Maps pending requests to FollowRequestResponse with the requester's actual username	Returns populated page
`getPending – requester deleted`	If the requester account is deleted (not found), uses the placeholder '[deleted]'	Returns page with [deleted] username

6. UserSettingsServiceImplTest

Class — service/UserSettingsServiceImplTest.java

Tests UserSettingsServiceImpl, which manages per-user notification and privacy settings with soft defaults.

Test Cases

Test Method	What It Verifies	Expected Outcome
`getOrDefault – settings exist`	Returns stored settings (readReceipts=false, onlineStatus=true, level=MENTIONS)	Returns stored settings
`getOrDefault – no settings`	Returns defaults: readReceipts=true, onlineStatus=true, notificationLevel=ALL	Returns defaults
`getSettings – found`	Returns the UserSettingsResponse for the given userId	Returns UserSettingsResponse
`getSettings – not found`	Throws ResourceNotFoundException when settings do not exist	throws ResourceNotFoundException
`initDefaults – already exist`	Returns existing settings without calling save	Returns existing, no save
`initDefaults – no settings`	Creates and saves default settings (readReceipts=true, level=ALL)	Default settings saved
`updateSettings – partial request`	Only non-null fields are applied; null fields remain unchanged; verified via argThat	Partial update applied
`updateSettings – all-null request`	When all request fields are null, settings are saved unchanged	No fields modified
`updateSettings – no existing settings`	Falls back to defaults before applying the changes	Defaults created then updated

7. UserHandlerTest

Class — web/UserHandlerTest.java

Tests the reactive WebFlux HTTP layer using WebTestClient bound directly to the router function. A UserPrincipal is injected into the reactive context via a WebFilter, so no Spring Security configuration is required.

Test Cases

Test Method	What It Verifies	Expected Outcome
`GET /api/v1/users/{username} - found`	Returns 200 OK with success=true and username in the data object	200 OK
`GET /api/v1/users/{username} - not found`	ResourceNotFoundException from the service propagates as 404	404 Not Found
`GET /api/v1/users/me`	Reads the principal from context, calls getById, returns 200 with profile data	200 OK
`PUT /api/v1/users/me`	Accepts a JSON body, delegates to updateProfile, returns 200	200 OK
`PUT /api/v1/users/me/avatar`	Extracts mediaId from body, calls updateAvatar, returns 200	200 OK
`PUT /api/v1/users/me/background`	Extracts mediaId from body, calls updateBackground, returns 200	200 OK
`PUT /api/v1/users/me/settings`	Accepts a JSON settings body, delegates to updateSettings, returns 200	200 OK
`GET /api/v1/users/{userId}/followers`	Calls getFollowers(userId, null, 20) and returns 200 with an items array	200 OK
`GET /api/v1/users/{userId}/following`	Calls getFollowing(userId, null, 20) and returns 200	200 OK
`POST /api/v1/users/{userId}/follow`	Calls follow(principal, targetId) and returns 204 No Content	204 No Content
`DELETE /api/v1/users/{userId}/follow`	Calls unfollow(principal, targetId) and returns 204 No Content	204 No Content
`GET /api/v1/users/me/follow-requests`	Calls getFollowRequests(principal) and returns 200	200 OK
`PUT /api/v1/users/me/follow-requests/{reqId}`	Passes the action string to respondFollowRequest and returns 204	204 No Content
`GET /api/v1/users/me/history`	Calls getHistory(principal) and returns 200	200 OK
`POST /api/v1/users/me/verify`	Parses the verification body and calls submitVerification; returns 202 Accepted	202 Accepted

Test Method	What It Verifies	Expected Outcome
`GET /api/v1/users/search`	Reads q and size query params, calls searchUsers, returns 200 with items array	200 OK

8. UserGrpcServiceTest

Class — grpc/UserGrpcServiceTest.java

Tests UserGrpcService, which exposes internal user data to other microservices via gRPC/Protobuf. Each test wraps the proto request in Mono.just(...) as required by the reactive gRPC signature.

Test Cases

Test Method	What It Verifies	Expected Outcome
`findUserById – found`	Maps User entity to proto FindUserByIdResponse with correct userId and username	Returns proto response
`findUserById – not found`	Propagates ResourceNotFoundException when the user does not exist	throws ResourceNotFoundException
`findUsersByIds`	Loads all users by UUID list and returns a response with users count = 2	Returns list of 2 users
`checkUserBlocked – blocked`	BlockService returns true → proto isBlocked = true	isBlocked = true
`checkUserBlocked – not blocked`	BlockService returns false → proto isBlocked = false	isBlocked = false
`getUserSettings`	Maps UserSettingsResponse to proto with readReceipts, onlineStatus and notificationLevel	Correct proto mapping
`getNotifPreferences – user found with tokens`	Maps FCM/APNS tokens and push/email flags from User entity	Tokens and flags populated
`getNotifPreferences – user not found`	Falls back to a default proto response with pushEnabled=true, emailEnabled=true and empty token strings	Defaults returned
`getNotifPreferences – null tokens`	null fcmToken and apnsToken are converted to empty strings to avoid proto null exceptions	Empty strings for null tokens

9. UserEventPublisherTest

Class — kafka/UserEventPublisherTest.java

Tests UserEventPublisher, which serialises domain events as Avro SpecificRecords and sends them to the reactive Kafka producer. The Kafka template is mocked to return Mono.just(senderResult).

Test Cases

Test Method	What It Verifies	Expected Outcome
`publishProfileUpdated`	Sends to topic user.profile.updated with the user UUID as the message key	Correct topic and key
`publishAvatarChanged – with URL`	Sends to user.avatar.changed with the user UUID as key	Correct topic and key
`publishAvatarChanged – null URL`	A null avatarUrl is defaulted to empty string before building the Avro record	No NullPointerException
`publishBackgroundChanged – with URL`	Sends to user.background.changed with the user UUID as key	Correct topic and key
`publishBackgroundChanged – null URL`	A null backgroundUrl is defaulted to empty string	No NullPointerException
`publishFollowed`	Sends to user.followed with the follower UUID as key, including follower username	Correct topic and key
`publishFollowed – null username`	A null followerUsername is defaulted to empty string	No NullPointerException
`publishUnfollowed`	Sends to user.unfollowed with the follower UUID as key	Correct topic and key

10. DtoValidationTest

Class — validation/DtoValidationTest.java

Tests Jakarta Bean Validation constraints on request DTOs using a plain Validator (no Spring context). Violations are inspected by property path name.

UpdateProfileRequest Constraints

Test Method	What It Verifies	Expected Outcome
`displayName > 100 chars`	A displayName exceeding 100 characters produces a violation on the displayName field	Violation on displayName
`bio > 500 chars`	A bio exceeding 500 characters produces a violation on the bio field	Violation on bio
`websiteUrl > 200 chars`	A websiteUrl exceeding 200 characters produces a violation on the websiteUrl field	Violation on websiteUrl
`location > 100 chars`	A location exceeding 100 characters produces a violation on the location field	Violation on location
`valid data`	A fully populated valid request produces zero violations	No violations
`all null fields`	All-null is valid (patch semantics) — zero violations	No violations

VerificationRequest Constraints

Test Method	What It Verifies	Expected Outcome
`blank documentMediaId`	@NotBlank fires when documentMediaId is an empty string	Violation on documentMediaId
`invalid type`	@Pattern rejects types not in the allowed set (IDENTITY, BUSINESS)	Violation on type
`IDENTITY type`	type=IDENTITY passes all constraints	No violations
`BUSINESS type`	type=BUSINESS passes all constraints	No violations
`both fields blank`	Both @NotBlank (documentMediaId) and @NotBlank + @Pattern (type) fire — at least 2 violations	2+ violations
`null documentMediaId`	@NotBlank rejects null as well as empty strings	Violation on documentMediaId